Sprint 4 Plan

SlugFound

CSE 115

Sprint Completion Date: TBD

Revision: 1.0

Revision Date: May 24, 2025

1. Sprint Goal

Polish and harden the SlugFound platform by delivering four targeted improvements: a fully fleshed-out item detail page that consolidates all item info in one place, a complete edit and delete workflow so owners can manage their own posts, a claim/resolve status flow that lets users close the loop on returned items. By the end of this sprint, SlugFound should feel production-ready — with clear ownership flows, reliable state management, and out-of-app engagement hooks.

2. Task Listing (by User Story)

Total estimated hours this sprint: 44 hours

User Story 4.1 [High] [5 pts]

"As a user, I want to tap on any item listing and see a full detail page with all available information — photos, description, location map, poster info, and a message button — so I can quickly decide whether the item is mine and contact the poster."

Specifications & Requirements

- Route: Dynamic Next.js route at /items/[id]. Fetch full item row from Supabase including joined profiles data (display_name, avatar_url) and lat/lng for map.
- Layout: Full-width image header (with placeholder if no image_url). Below: title, type badge (Lost/Found), category chip, status badge (Active/Claimed/Resolved), date posted, poster name with initials avatar.
- Description block: Full description text, location pill, and a react-leaflet mini-map centered on the item's coordinates (only rendered if lat/lng are non-null).
- Action bar: 'Message Poster' button (visible to authenticated non-owners; initiates or resumes a conversation from Sprint 3). 'Edit Post' and 'Mark as Claimed/Resolved' buttons visible to the item's owner only.
- SEO / meta: Set Next.js page <title> and <meta description> dynamically from item title and description.
- 404 handling: If item id does not exist, render a friendly not-found page with a link back to listings.
- Loading: Show skeleton loaders while Supabase fetch is in progress.

Tasks

- Create dynamic /items/[id] route with Supabase item + profile join fetch and skeleton loader *(2 hrs)*
- Build detail page layout: image header, title/badges block, description, location pill *(3 hrs)*
- Embed react-leaflet read-only pin map on detail page when lat/lng is available *(1 hr)*
- Implement owner-only action bar (Edit, Mark as Claimed/Resolved) and non-owner Message Poster button *(2 hrs)*
- Add dynamic <title>/<meta> tags and 404 not-found page *(1 hr)*

Total for User Story 4.1: 9 hours

Definition of Done

- Navigating to /items/[id] renders the full item detail page with no hardcoded data
- Poster name, avatar initials, and all item fields are pulled live from Supabase
- Map renders correctly when coordinates exist; section is hidden when they do not
- Message Poster button is hidden for the item's own owner and visible to all other authenticated users
- Edit and Mark as Claimed/Resolved buttons are visible only to the item's owner
- Entering a non-existent item ID shows the 404 page, not a crash
- All code merged to main on GitHub with a passing build

User Story 4.2 [High] [5 pts]

"As the owner of a post, I want to edit or delete it so that I can correct mistakes, add new information, or remove a post when the item has been handled outside the app."

Specifications & Requirements

- Edit form: Pre-populate the existing post form with current values from Supabase. Support editing title, description, category, location, and photo. Changing the photo uploads a new file to Supabase Storage and deletes the old one.
- Update call: supabase.from('items').update().eq('id', itemId) — only permitted when auth.uid() matches user_id (enforced by existing RLS).
- Delete: Show a confirmation modal ('Are you sure? This cannot be undone.') before calling .delete(). On confirm, delete the item row and its Storage image (if any), then redirect to the appropriate listings page.
- Access control: Edit and delete entry points (buttons on detail page and account page post list) are rendered only for the authenticated owner. Direct URL access to /items/[id]/edit by a non-owner returns a 403 message.
- Updated_at: Trigger a Supabase SQL function to auto-update the updated_at timestamp on every UPDATE.
- Optimistic UI: Show a loading spinner on the save button during the update call; show a success toast on completion.

Tasks

- Create /items/[id]/edit route that pre-populates form with existing item data *(2 hrs)*
- Implement Supabase update() call with image swap logic (upload new, delete old from Storage) *(2 hrs)*
- Build delete confirmation modal and wire to Supabase delete() with Storage cleanup *(2 hrs)*
- Add 403 guard on /items/[id]/edit for non-owners *(1 hr)*
- Write Supabase SQL trigger function to auto-update updated_at on item row update *(1 hr)*

Total for User Story 4.2: 8 hours

Definition of Done

- Owner can edit any field on their post and changes persist to Supabase after saving
- Editing a photo uploads the new image and removes the old one from Storage
- Delete requires confirmation modal; after confirmation the item row and image are removed
- A non-owner visiting /items/[id]/edit directly sees a 403 message, not the form
- updated_at column reflects the time of the most recent edit in the Supabase dashboard
- All code merged to main on GitHub with a passing build

User Story 4.3 [High] [5 pts]

"As the owner of a post, I want to mark it as Claimed or Resolved so that other users know the item has been returned, and the listing is visually distinguished from active posts."

Specifications & Requirements

- Status values: items.status enum already supports 'active' | 'claimed' | 'resolved' from Sprint 2. This story builds the UI and business logic to transition status.
- Owner controls: On the item detail page, the action bar shows two buttons for the owner: 'Mark as Claimed' (transitions active → claimed) and 'Mark as Resolved' (transitions active or claimed → resolved). Once resolved, no further transitions are allowed.
- Visual treatment: Item cards on listings pages show a muted overlay + 'Claimed' or 'Resolved' badge when status is not active. These items sort to the bottom of the list (active items first, ordered by created_at desc within each group).
- Filter: Add an 'Active only' toggle to the listings pages (default on) so users can choose to show or hide claimed/resolved items.
- Account page stats: Update the reunited count stat card (from Sprint 2) to accurately reflect items with status='claimed' or 'resolved'.
- Supabase: Status update via supabase.from('items').update({ status }).eq('id', itemId), protected by existing RLS (owner only).

Tasks

- Add Mark as Claimed and Mark as Resolved buttons to item detail page action bar with transition logic *(2 hrs)*

- Update item cards on listings pages to render muted overlay and status badge for non-active items *(2 hrs)*
- Implement active-first sorting logic in Supabase query (active items before claimed/resolved) *(1 hr)*
- Add Active Only toggle filter to Found and Lost pages *(1 hr)*
- Update Account page reunited stat to query claimed + resolved count *(1 hr)*

Total for User Story 4.3: 7 hours

Definition of Done

- Owner can transition a post from active → claimed or active → resolved from the detail page
- Resolved posts cannot be transitioned further; both buttons are disabled/hidden once resolved
- Claimed and resolved items display a muted overlay and badge on listing cards
- Active items always sort above claimed/resolved items on listing pages
- Active Only toggle hides claimed and resolved items when enabled (default on)
- Account page reunited count reflects actual claimed + resolved count from Supabase
- All code merged to main on GitHub with a passing build

User Story 4.4 [Medium] [5 pts]

"As a user, I want to send images in messages so that I can better describe or identify a lost or found item when communicating with another user."

Specifications & Requirements

- Image upload: Add an attachment button next to the message input that opens a file picker accepting image/jpeg, image/png, image/webp only, max 5MB
- Preview: Show a thumbnail preview of the selected image above the input with an X button to remove it before sending
- Storage: Upload images to Supabase Storage at path `message-images/<user_id>/<uuid>.<ext>` and store the public URL in the existing `image_url` column on the messages table
- Display: Render image messages as tappable thumbnails inside the chat bubble. If a message has both text and an image, show both
- The user can send a message with just an image, just text, or both. Send button is disabled if both are empty
- Storage RLS: Add a policy to allow authenticated users to upload only to their own `message-images/<user_id>/` folder

Tasks

- Add attachment button and file input with image type/size validation to chat input (1 hr)
- Implement thumbnail preview with remove button above input (1 hr)
- Upload image to Supabase Storage and get public URL on send (2 hrs)
- Update message send logic to include image_url in the insert (1 hr)
- Render image thumbnails in chat bubbles with text fallback (2 hrs)
- Add Storage RLS policy for message-images folder (1 hr)

Total for User Story 4.4: 8 hours

Definition of Done

- User can select an image and see a preview before sending
- Image is uploaded to Supabase Storage and URL saved to messages table
- Image renders inside the chat bubble for both sender and recipient
- Messages with only an image send successfully with no text required
- Files over 5MB or wrong type are rejected with a clear error toast
- Users can only upload to their own folder (RLS enforced)
- All code merged to main on GitHub with a passing build

User Story 4.5 [Medium] [3 pts]

"As a user, I want to edit my profile — including my display name and avatar photo — so that other users see accurate and personalized information when I post or message them."

Specifications & Requirements

- Edit fields: Allow user to update display_name (max 40 chars, required) and upload a profile photo (JPG/PNG/WebP, max 2MB) stored in a new Supabase Storage bucket 'avatars'.
- Avatar display: If avatar_url is set, render the image in the sidebar, Account page header, and message conversation list instead of the initials fallback. Initials fallback remains for users without an avatar.
- Inline edit: On the Account page, clicking 'Edit Profile' expands an inline form (no separate page). Saving calls supabase.from('profiles').update() and refreshes the AuthContext.
- Propagation: After a display_name or avatar_url update, all previously rendered item cards and message threads should reflect the new values on next fetch (no stale data from cached joins).
- Validation: display_name must be non-empty and max 40 chars. Avatar upload fails gracefully with a toast if file type or size is invalid.

Tasks

- Create avatars Storage bucket with authenticated-user upload policy and 2MB size limit *(1 hr)*
- Build inline edit form on Account page with display_name field and avatar upload input *(2 hrs)*
- Implement Supabase profiles.update() call with avatar upload and AuthContext refresh *(2 hrs)*
- Update sidebar, item cards, and message threads to render avatar image when avatar_url is set *(1 hr)*

Total for User Story 4.5: 6 hours

Definition of Done

- User can update their display name and it immediately reflects in the sidebar and Account page
- User can upload a profile photo; it replaces the initials avatar across the sidebar, item cards, and messages
- Saving with an empty display name shows a validation error and does not call Supabase
- Uploading an oversized or wrong-format file shows a toast error and does not crash the form
- All code merged to main on GitHub with a passing build

User Story 4.6 [Low] [2 pts]

“As any user, I want to flag an item post as inappropriate so that the community stays safe and spam or offensive listings can be reviewed.”

Specifications & Requirements

- Reports table: Create a public.reports table with columns: id (uuid, PK), item_id (FK to items.id), reporter_id (FK to profiles.id), reason (enum: 'spam' | 'offensive' | 'duplicate' | 'other'), notes (text, nullable, max 300 chars), created_at (timestamptz). Unique constraint on (item_id, reporter_id) — one report per user per item.
- Report UI: Add a ‘Report’ overflow menu option on the item detail page (hidden for item owners). Clicking opens a small modal with a reason dropdown and optional notes field.
- Submission: Insert a row into reports. Show a success toast (‘Thanks — our team will review this.’). If the user has already reported the item, show an informational message instead of the form.
- RLS: Authenticated users can INSERT reports where reporter_id = their own auth ID. No user can SELECT, UPDATE, or DELETE reports (admin-only access via Supabase service role key in future).
- Threshold flag (stretch): If an item accumulates 3 or more reports, add a reported_flag boolean column to items and set it to true via a Supabase SQL trigger. Items with reported_flag = true are hidden from listings but still accessible via direct URL with a warning banner.

Tasks

- Write CREATE TABLE migration for reports table with RLS policies *(1 hr)*
- Build report modal UI with reason dropdown, optional notes, and duplicate-report guard *(2 hrs)*
- Wire report form to Supabase reports.insert() with success/duplicate toast handling *(1 hr)*
- (Stretch) Add reported_flag column to items and SQL trigger to set it after 3 reports *(1 hr)*

Total for User Story 4.6: 5 hours

Definition of Done

- Any authenticated non-owner can open the Report modal on an item detail page and submit a report

- A row is inserted into the reports table with the correct item_id, reporter_id, and reason
- Submitting a second report on the same item shows an informational message, not a duplicate row
- Item owners do not see the Report option on their own posts
- RLS prevents any user from reading others' report submissions in the Supabase SQL editor
- All code merged to main on GitHub with a passing build

User Story 4.7 [Medium] [3 pts]

"As a user browsing lost or found items, I want to switch to a map view so that I can visually see where items were found or lost on the UCSC campus."

Specifications & Requirements

- Map toggle: Add a Map / List toggle button group to both the Found Items and Lost Items pages, consistent with the existing search/filter bar design.
- Map library: Use the same react-leaflet + OpenStreetMap setup from US 3.2. Initialize the map centered on UCSC's coordinates (36.9916° N, 122.0583° W) at zoom level 15.
- Item pins: Render a map pin for each item that has lat/lng data. Items without coordinates are excluded from map view. Show the total count of items with coordinates vs. total items in an info bar above the map.
- Pin popup: Clicking a pin opens a popup with the item photo thumbnail, title, category badge, date, and a 'View Item' link that opens the item detail page.
- Filters apply to map: Applying the search bar or category filter updates the map pins in real-time, the same as it updates the list view.
- Responsive: Map view fills available width and is at least 500px tall on desktop and full-screen on mobile.

Tasks

- Add Map / List toggle control to Found and Lost pages (1 hr)
- Render react-leaflet map with UCSC center and item pins from Supabase query results (2 hrs)
- Build pin popup component with item photo, title, category, and View Item link (1 hr)
- Ensure search and category filter updates apply to map pins in real-time (1 hr)
- Add info bar showing count of geotagged items vs. total items (1 hr)

Total for User Story 3.5: 6 hours

Definition of Done

- Found Items and Lost Items pages each have a working Map / List toggle
- Map renders centered on UCSC campus with pins for all geotagged items of the correct type
- Clicking any pin opens a popup with correct item details and a working View Item link
- Applying search or category filters removes non-matching pins from the map in real-time
- Items without lat/lng coordinates are gracefully excluded from map view without errors
- All code merged to main on GitHub with a passing build

3. Team Roles

Team Member	Email	Role(s)
Bardia Nasab	bnasab@ucsc.edu	Scrum Master
Colin Posat	cposat@ucsc.edu	Developer, UI/UX Lead
Sam Madinya	smadinya@ucsc.edu	Developer
Aidan Nguyen	anguy480@ucsc.edu	Developer

4. Initial Task Assignment

Team Member	User Story	Initial Task
Bardia Nasab	US 4.1	Build item detail page layout and wire to Supabase item fetch
Colin Posat	US 4.2	Build edit post form with pre-populated fields and Supabase update call
Sam Madinya	US 4.3	Design claim/resolve status flow and write Supabase migration
Aidan Nguyen	US 4.4	Set up messaging images

5. Initial Scrum Board

The Sprint 4 scrum board is managed in Jira

6. Initial Burnup Chart

The Sprint 4 burnup chart is tracked in Jira and will be updated as tasks are completed.

Total sprint capacity: 25 story points across 44 estimated hours.

7. Scrum Meeting Times

Meeting #	Day	Time	Location	TA/Tutor Visit?
1	Mondays	12:00 – 1:00 PM	Zoom	Yes — Lab session
2	Wednesdays	5:30 – 6:30 PM	TBD	No

3	Fridays	12:00 – 1:00 PM	TBD	No